

CO₄-AT₁ - Comparative Analysis

Name :- P. Rohini

Course Name :- Data Structures

Reg No :- M2524147

Date :- 14-08-2026

Course Code :- CSA0307

1. GPU Bitonic sort (CUDA) vs CPU parallel Quicksort (64 cores)

Introduction :

Sorting 1 billion integer is a computationally intensive task. GPUs and CPUs use different parallelization strategies to improve performance. GPU Bitonic Sort exploits massive parallelism, while CPU parallel Quicksort utilizes multiple CPU cores.

Comparison :

Features	GPU Bitonic sort (CUDA)	CPU parallel Quicksort
Time complexity	$O(\log^2 n)$ parallel stages	$O((n \log n)/p)$, where $P = 64$ Cores
Parallelism	Thousands of GPU threads	64 CPU cores
Memory usage	Shared Memory + Global Memory	Cache + Main Memory
Data Access	Highly parallel	Moderate parallelism
Branching	Minimal warp divergence	Recursive branching
Transfer Cost	PCIe transfer required	No transfer overhead.

Memory Hierarchy Analysis

GPU

- * Shared Memory : Very fast, used for sorting small blocks.
- * Global Memory : Large but slower
- * Data is moved between shared and global memory during sorting

CPU

- * Uses L1, L2 and L3 caches.
- * Better for irregular memory access patterns
- * No explicit memory transfer required

PCIe Transfer Overhead

For 1 billion integers:

* Integer size = 4 bytes

* Total data = 4 GB

Transfer Time:

$$\text{Transfer Time} = \frac{4 \text{ GB}}{16 \text{ GB/s}}$$

≈ 0.25 Seconds each way

Round trip:

≈ 0.5 seconds.

This overhead can dominate performance for smaller data sets.

Small Inputs (< 10 Million integers).

- * CPU is usually faster
- * PCIe transfer overhead dominates GPU execution.

Medium Inputs (10 million - 100 millions)

* performance becomes comparable.

Large Input (> 100 million)

* GPU becomes advantageous

* Massive parallelism outweighs transfer costs.

For 1 Billion Integers.

* GPU generally provides $2\times - 10\times$ speedup compared to CPU parallel quicksort.

Conclusion:

For sorting 1 billion integers, GPU Bitonic sort benefits from massive parallelism and high memory bandwidth. Although PCIe transfer introduces overhead, GPUs become advantages for datasets large than approximately 100 million integers.

2. Linear probing Vs Cuckoo Hashing (100 Million Keys, 90% Lookups, 10% Inserts).

Introduction:

Hash tables are widely used for fast data retrieval. Linear probing and cuckoo hashing are two popular collision-resolution techniques.

Comparison.

Features	Linear probing	Cuckoo Hashing
Lookup Time	$O(1)$ average	$O(1)$ worst-case
Insert Time	$O(1)$ average	$O(1)$ average

Worst - Case Lookup	$O(n)$	$O(1)$
Cache performance	Excellent	Moderate
Memory Usage	Low	Higher
Rehashing Cost	Moderate	High
Insertion Failure	Rare	Possible.

Read-Heavy Workload Analysis

Workload

- * 100 Million Keys
- * 90% Lookups
- * 10% Inserts

Worst - Case Lookup Guarantees

Linear probing

Worst Case: $O(n)$

Reason: Long clusters force many probes.

Cuckoo Hashing

Worst Case: $O(1)$

Reason: Each key can only be in one of two possible locations.

Insertion Failures.

Linear probing

- * Usually Succeeds if free slots exist.

* performance degrades as load factor increases.

Cuckoo Hashing

* Cycles may occur during relocation

* Insertion can fail even when space exists.

Rehashing Costs

Linear probing

Rehash Cost : $O(n)$

occurs when table becomes too full.

Cuckoo Hashing

Rehash Cost : $O(n)$

However, rehashing is more expensive because all keys must be relocated using new hash function.

Conclusion:

For a read-heavy workload with 100 million keys and 90% lookups, Linear probing is generally preferred because of its superior cache locality and faster practical performance. Cuckoo hashing provides guaranteed $O(1)$ lookups but incurs higher insertion and rehashing costs. Therefore, Linear probing is the better choice for this workload, while Cuckoo hashing is suitable when worst case lookup guarantees are critical.